MATH423: Introduction to Computational Topology

Why Simplicial Complexes? Computational Topology in Practice

Mehmetcik Pamuk (mpamuk@metu.edu.tr)
Department of Mathematics, Middle East Technical University

2026-27 FALL

4 Interlude: Why Simplicial Complexes? Computational Topology in Practice

A question that comes up every year, especially from students with a programming background, is: *why do we insist on simplicial complexes?* Why not work directly with the point cloud, or with a graph, or with a smooth surface? This interlude answers that question from the computer's point of view, and then surveys applications in which simplicial complexes, and the topology computed from them, solve real problems. The mathematics needed for the applications is developed in the rest of the course: homology (Week 5), algorithms (Week 6) and persistence (Weeks 7–8). Here the goal is to see where we are heading.

4.1 The Computer's Point of View

A computer cannot store a continuous space

The torus T^2, the sphere S^2, or the union of balls around a point cloud all have uncountably many points. A computer can only store a finite amount of data, so it can never hold such a space directly. It can, however, hold a finite description from which the space can be reconstructed. A finite simplicial complex is exactly such a description: the complex K is a finite list, while the polyhedron $|K|$ is the continuous space it describes. All of the topology of $|K|$ (connectedness, Euler characteristic, orientability, and later homology) can be computed from the finite list K alone. This is the sense in which triangulations make topology computable.

Example 4.1. The torus with the 9-vertex triangulation of Week 2 is stored as the following list of 18 triangles. As is usual in programming, the vertices are numbered from 0, so the vertex labelled k in the figure of Week 2 is vertex $k-1$ here.

```
torus = [(0,1,4), (0,4,3), (1,2,5), (1,5,4), (2,0,3), (2,3,5),
         (3,4,7), (3,7,6), (4,5,8), (4,8,7), (5,3,6), (5,6,8),
         (6,7,1), (6,1,0), (7,8,2), (7,2,1), (8,6,0), (8,0,2)]
```

These 18 triples of integers *are* the torus, as far as a computer is concerned. Every edge and vertex is recovered as a subset of a triangle, and the Euler characteristic is obtained by counting: $9 - 27 + 18 = 0$. The real projective plane needs only the 10 triangles on 6 vertices from Worksheet 1, Problem 7.

Simplices are determined by their vertices

A k-simplex is determined by its $k+1$ vertices, and its faces are exactly the non-empty subsets of the vertex set. This has several practical consequences.

- **Compact storage.** A complex is stored as a list of vertex sets, and in practice as the list of its maximal simplices only; all other simplices are subsets of these.
- **Faces are free.** Enumerating the faces of a simplex is enumerating subsets of a small set. There is no geometry to compute.
- **No floating point topology.** The coordinates of the vertices may be rounded, but the combinatorial structure is exact. Two programs reading the same list of triangles compute the same Euler characteristic, with no numerical tolerance involved.
- **Uniform formulas.** Because every face of a simplex is again a simplex, one formula (the boundary operator of Week 5) works in every dimension. With arbitrary polygons or polyhedral cells, each cell type needs its own bookkeeping.

This is also why simplicial complexes are the standard format in computer graphics. A triangle mesh in the widely used OBJ file format is a list of vertex coordinates followed by a list of faces, each face given by vertex indices:

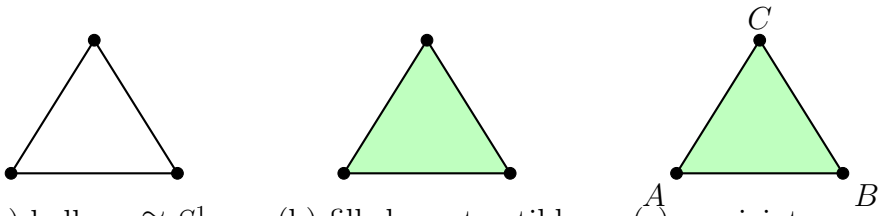
```
v 0 0 0
v 1 0 0
v 0 1 0
v 0 0 1
f 1 3 2
f 1 2 4
f 1 4 3
f 2 3 4
```

The `f` lines are an abstract simplicial complex (here the boundary of the tetrahedron, which triangulates S^2), and the `v` lines give its geometric realization in $\mathbb{R}^3$. Graphics cards draw triangles; finite element software for engineering simulations works with triangle and tetrahedral meshes. Simplicial complexes are already everywhere in computing.

Why not just a graph?

Computer science students are used to graphs, and a graph is a 1-dimensional simplicial complex. The trouble is that a graph cannot tell the difference between a hole and a filled region.

Example 4.2. Consider three researchers A, B, C. In situation (i), each pair has written a paper together, but the three never wrote a paper as a group. In situation (ii), they wrote a single paper with all three as authors. The co-authorship *graph* is a triangle in both cases. The co-authorship *complex*, in which every paper with $k+1$ authors contributes a k-simplex, is a hollow triangle in case (i) and a filled triangle in case (ii). Group interactions (papers, chemical reactions involving several molecules, group conversations, neurons firing together) are naturally simplices, not edges. Network science calls these *higher-order interactions*, and simplicial complexes are one of the standard models for



(a) hollow: $\cong S^1$ (b) filled: contractible (c) one joint paper

Figure 10: The complexes (a) and (b) have the same 1-skeleton, so as graphs they are identical. Only the 2-simplex distinguishes a loop from a filled triangle.

them. For example, Benson, Abebe, Schaub, Jadbabaie and Kleinberg studied, in many real data sets, how often a configuration like (i) is later "closed" into a configuration like (ii), and used this to predict future group interactions.

The Vietoris-Rips complex is a compromise between the two worlds: it is completely determined by a graph (Week 3), so one only stores the edges, but its higher simplices fill in triangles, tetrahedra and so on, so that it can distinguish holes from filled regions.

Only distances are needed

The Vietoris-Rips complex only uses the pairwise distances $d(x_i, x_j)$. No coordinates, no ambient $\mathbb{R}^n$, and no notion of angle or straight line are needed. Consequently, it can be built for data that do not live in Euclidean space at all:

- DNA or protein sequences, with the number of positions in which two sequences differ (Hamming distance);
- text documents, with a distance computed from word frequencies;
- images, sound recordings or time series, with any distance a practitioner finds meaningful;
- nodes of a network, with the length of the shortest path between them.

In all these cases the input is simply an $N \times N$ matrix of distances, and the Rips complex turns it into a space whose shape can be studied.

Topology that a computer can actually decide

It is a remarkable fact that many natural topological questions are *algorithmically undecidable*: no computer program can answer them correctly for all inputs.

- There is no algorithm that decides whether two given triangulated 4-manifolds are homeomorphic (A. A. Markov, 1958).
- There is no algorithm that decides whether a given finite 2-dimensional simplicial complex is simply connected, that is, whether every loop in it can be contracted to a point. This follows from the undecidability of the triviality problem for finitely presented groups (Adian–Rabin), since every finite group presentation arises as the fundamental group of a finite 2-dimensional complex.

In contrast, *homology*, which we will define in Week 5, can always be computed. For a finite simplicial complex, homology is obtained from the boundary matrices by Gaussian elimination, and these matrices are sparse, with exactly $k+1$ nonzero entries in the column of a k-simplex. This is the main reason computational topology is built on homology: it is a strong enough invariant to detect holes in every dimension, and weak enough to be computable by linear algebra. Simplicial complexes are the input format for which this linear algebra is simplest.

Even before homology, some of this is familiar from algorithms courses. Counting the connected components of a complex only needs its 1-skeleton and a graph search or a union-find structure. For a connected graph with V vertices and E edges, the number of independent cycles is $E-V+1$. Homology extends this kind of counting to 2-dimensional voids, 3-dimensional cavities, and beyond.

The price: the size of the complex

Simplicial complexes have one serious drawback. If N points are all within distance 2ϵ of each other, then VR_ϵ is the full simplex on N vertices and has $2^N - 1$ simplices. Even when we stop at a fixed dimension the numbers grow quickly. For $N = 1000$ points the full simplex has

$$\binom{1000}{2} = 499{,}500 \text{ edges}, \qquad \binom{1000}{3} = 166{,}167{,}000 \text{ triangles},$$
$$\binom{1000}{4} = 41{,}417{,}124{,}750 \approx 4.1 \times 10^{10} \text{ tetrahedra}.$$

Much of the research in computational topology is about keeping complexes small without changing their topology. For example, the **alpha complex** of a point set in the plane is a subcomplex of its Delaunay triangulation, which has at most $2N - 5$ triangles for $N \geq 3$ points not all on a line, and it is still homotopy equivalent to the union of balls. Another strategy is simplification: removing simplices in a way that is guaranteed not to change the topology, as in discrete Morse theory (Weeks 10–11) and in the edge contractions of Application 4.3.2 below.

Remark 4.3. Truncation must be done with care. Let X be the four corners of the unit square and $\epsilon = 0.75$, so that $VR_\epsilon(X)$ is the full 3-simplex, with $\chi = 4 - 6 + 4 - 1 = 1$. A program that only builds simplices up to dimension 2 produces the boundary of the tetrahedron instead, which is a sphere with $\chi = 2$: the truncation has created a 2-dimensional void that is not there. If we want correct information about holes of dimension k, we must build the complex up to dimension $k + 1$.

4.2 A First Program

The following Python code builds the Vietoris-Rips complex of a finite point set up to a given dimension and computes its Euler characteristic. It only uses the standard library.

```
from itertools import combinations
from math import dist

def rips_complex(points, eps, max_dim=2):
    """All simplices of VR_eps(points) of dimension <= max_dim."""
```

```
    n = len(points)
    edges = {(i, j) for i, j in combinations(range(n), 2)
             if dist(points[i], points[j]) <= 2 * eps}
    K = [(i,) for i in range(n)] + sorted(edges)
    for k in range(3, max_dim + 2):          # k = number of vertices
        K += [s for s in combinations(range(n), k)
              if all(e in edges for e in combinations(s, 2))]
    return K

def euler_characteristic(K):
    return sum((-1) ** (len(s) - 1) for s in K)
```

For the four corners of the unit square, `rips_complex(square, 0.6, 3)` returns 8 simplices with $\chi = 0$ (the hollow square), and `rips_complex(square, 0.75, 3)` returns all 15 simplices of the tetrahedron with $\chi = 1$, as in the table of Week 3. The code is deliberately naive: it tests every k-element subset of the points, which is far too slow for large data sets (see Programming Exercise 1).

The next function checks the condition from the Lemma of Week 2 for a closed surface, namely that every edge lies in exactly two triangles.

```
from collections import Counter

def edge_counts(triangles):
    """How many triangles contain each edge."""
    return Counter(frozenset(e) for t in triangles
                   for e in combinations(t, 2))

def is_closed_surface(triangles):
    return all(c == 2 for c in edge_counts(triangles).values())
```

It returns `True` for the torus list above, for the 6-vertex $\mathbb{R}P^2$, and for the OBJ tetrahedron; it returns `False` as soon as one triangle is deleted, since three edges then lie in only one triangle. Note that this test is necessary but not sufficient for a closed surface: two tetrahedron boundaries sharing a single vertex pass it, but the shared vertex has a link consisting of two circles (compare Worksheet 1, Problem 6).

4.3 Applications

In each application below, ask yourself three questions: *What is the space? What simplicial complex is built from the data? What does its topology tell us?* The answers are collected in a table at the end of the section.

4.3.1 Meshes, 3D Printing and Computer Graphics

A 3D printer can only print a solid object if the triangle mesh describing its surface is a closed surface: it must have no holes, and no edge may be shared by three or more triangles. Slicing software checks exactly the condition of the Lemma of Week 2. An edge contained in only one triangle signals a hole in the surface, and an edge contained in three or more triangles is called a *non-manifold edge*. Both are reported as errors and must be repaired before printing.

Two more pieces of our theory appear directly.

- **Genus.** For a closed orientable mesh, the genus is $g = (2-\chi)/2$, which is obtained by counting vertices, edges and triangles. A scanned coffee mug should have $\chi = 0$ and $g = 1$; a value $\chi = -2$ reveals an extra handle, typically a scanning artifact where two nearby parts of the surface were wrongly joined.

- **Orientation.** A closed surface embedded in $\mathbb{R}^3$ is always orientable, because a closed non-orientable surface such as the Klein bottle cannot be embedded in $\mathbb{R}^3$. Software orients triangles consistently, one neighbor at a time, so that every normal vector points outward. If this fails for a closed mesh, the mesh cannot be embedded, so it must intersect itself somewhere.

4.3.2 Mesh Simplification and the Link Condition

Meshes from scanners can have millions of triangles. A standard way to reduce their size is **edge contraction**: an edge $\{a, b\}$ is shrunk to a single point, and the triangles containing it disappear. Contracting the wrong edge, however, can change the topology, for example by pinching a thin handle. Dey, Edelsbrunner, Guha and Nekhayev proved that, for a triangulated surface, contracting $\{a, b\}$ preserves the topological type if and only if the **link condition**

$$\mathrm{lk}(a) \cap \mathrm{lk}(b) = \mathrm{lk}(\{a, b\})$$

holds, where links are as defined in Week 3. The test only involves the neighbors of a and b, so it is cheap to check.

Example 4.4.

- Let K be the boundary of the octahedron, with poles N, S and equator vertices $1, 2, 3, 4$ in cyclic order. Then $\mathrm{lk}(N)$ is the cycle $1-2-3-4-1$ and $\mathrm{lk}(1)$ is the cycle $N-2-S-4-N$. They have the vertices 2, 4 in common and no common edge, so $\mathrm{lk}(N) \cap \mathrm{lk}(1) = \{\{2\}, \{4\}\} = \mathrm{lk}(\{N, 1\})$. The link condition holds, and contracting $\{N, 1\}$ gives a triangulation of S^2 with 5 vertices.

- Let K be the boundary of the tetrahedron on $\{1, 2, 3, 4\}$. Then $\mathrm{lk}(1) \cap \mathrm{lk}(2)$ contains the edge $\{3, 4\}$, while $\mathrm{lk}(\{1, 2\}) = \{\{3\}, \{4\}\}$ does not. The link condition fails: contracting $\{1, 2\}$ would make the triangles $\{1, 3, 4\}$ and $\{2, 3, 4\}$ coincide, and the result is no longer a surface. This is consistent with the fact that 4 vertices is the minimum for a triangulation of S^2.

4.3.3 Digital Images and Character Recognition

A black-and-white image is a set of pixels, and each black pixel can be viewed as a closed unit square; the image is then a **cubical complex**, the square analogue of a simplicial complex. (Each square can be cut into two triangles if a simplicial complex is preferred.) The simplest topological features are the number b_0 of connected components and the number b_1 of holes, which we will later recognize as Betti numbers. In a typical sans-serif font:

Character	C	A	O	B	8	i	j	%
b_0 (components)	1	1	1	1	1	2	2	3
b_1 (holes)	0	1	1	2	2	0	0	2

These numbers do not change when a character is enlarged, slanted or written in a slightly different font, which makes them useful features for recognition. They are also coarse: A and O have the same Betti numbers, so topology must be combined with other information.

Working with closed squares also settles a classical ambiguity of digital images. If two black pixels touch only at a corner, are they connected? Different conventions ("4-connectivity" and "8-connectivity") give different answers, and using one convention for black pixels and the same one for white pixels leads to paradoxes such as a closed digital curve that does not separate its inside from its outside. In a cubical complex, two closed squares that share a corner intersect, so the question has a definite answer.

4.3.4 Coverage in Sensor Networks

Suppose many cheap sensors are scattered over a region to detect intruders, and each sensor monitors a disk of radius r_c around itself. The sensors do not know their positions (there is no GPS indoors, for instance), but each can detect which other sensors lie within communication distance r_b. Is the whole region monitored?

The available data are exactly the pairs of sensors within distance r_b, that is, the 1-skeleton of a Vietoris-Rips complex, and hence the whole Rips complex. Its triangles are covered by the sensors when the radii are suitably related:

Proposition 4.5. *Let T be a triangle in the plane whose three side lengths are at most r_b. If $r_c \geq r_b/\sqrt{3}$, then T is contained in the union of the three closed disks of radius r_c centered at its vertices.*

Proof for acute and right triangles. Let O be the circumcenter and R the circumradius; for an acute or right triangle O lies in T. The segments from O to the midpoints of the three sides cut T into three quadrilaterals, each containing exactly one vertex v of T. The four corners of the quadrilateral containing v are v itself, two midpoints at distance at most $r_b/2$ from v, and O at distance R from v. Since every side of T is a chord of the circumcircle, it has length at most $2R$, so all four corners are within distance R of v. A disk is convex, so the whole quadrilateral lies in the disk of radius R around v. By Jung's Theorem in the plane (Week 3), $R \leq r_b/\sqrt{3} \leq r_c$. The obtuse case is Programming Exercise 6. □

The constant $1/\sqrt{3}$ is the circumradius of the equilateral triangle with unit sides, the extremal case of Week 3. So under this assumption a region filled by Rips triangles is fully monitored, and the remaining question is whether the Rips complex has holes. De Silva and Ghrist turned this observation into a rigorous criterion: coverage of the region (away from its boundary) is guaranteed when a certain homology class of the Rips complex, relative to the sensors near the boundary, is nonzero. That criterion is computed from the communication graph alone, with no coordinates, by the matrix algorithms of Week 6.

4.3.5 Configuration Spaces of Robots

A robot arm in the plane consists of two links, attached at a fixed base and at an elbow, each joint rotating freely through a full turn. A position of the arm is given by two angles (θ_1, θ_2), each an element of the circle S^1. The set of all positions, the **configuration**

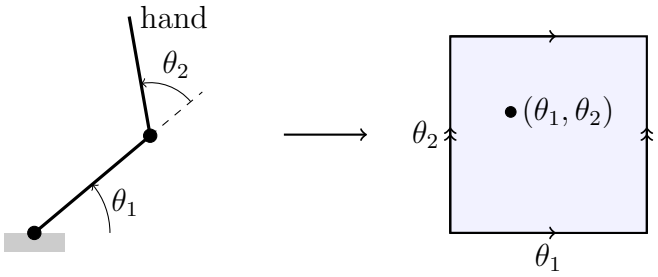


Figure 11: A two-link planar arm and its configuration space, the torus.

space of the arm, is therefore the torus $S^1 \times S^1 = T^2$, the square $[0, 2\pi]^2$ with opposite sides identified, exactly as in Week 2.

More complicated robots, several robots sharing a factory floor, or an arm moving among obstacles have more complicated configuration spaces. Motion planning means finding a path in the configuration space between two given configurations. Farber showed that the topology of the configuration space limits how well this can be done: a motion planning algorithm that depends continuously on the start and end configurations exists only if the configuration space is contractible. For the arm above it is a torus, which is not contractible, so every motion planner for it must have discontinuities, places where an arbitrarily small change in the input produces a very different path. Farber's *topological complexity* measures how many such discontinuities are unavoidable.

4.3.6 The Shape of Natural Images

Carlsson, Ishkhanov, de Silva and Zomorodian studied a large collection of small 3×3 pixel patches cut from photographs of natural scenes. Each patch is a point in $\mathbb{R}^9$. After normalizing for brightness and contrast and keeping only the densest regions of high-contrast patches, they built a simplicial complex from the resulting point cloud (a *witness complex*, a smaller relative of the Vietoris-Rips complex) and computed its persistent homology. The answer was a Klein bottle, the surface of Week 2. Patches showing an edge at different angles form one circle on it, and the Klein bottle organizes how these edge patches fit together with patches showing a thin line or bar. The appearance of this surface in image statistics was a surprise, and it was found by computing topology, not by looking at pictures.

4.3.7 Mapper: Seeing the Shape of High-Dimensional Data

The Mapper algorithm of Singh, Mémoli and Carlsson produces a small simplicial complex summarizing a large data set X. It is a direct application of the nerve construction of Week 3.

1. Choose a function $f : X \to \mathbb{R}$, such as a density estimate or the distance to some reference point.

2. Cover the range of f by overlapping intervals $I_1, \ldots, I_m$.

3. Split each preimage $f^{-1}(I_j)$ into clusters, using any clustering algorithm.

4. Output the **nerve** of the collection of all clusters: one vertex per cluster, and an edge whenever two clusters share a data point. Since consecutive intervals overlap, so do their clusters.

Using Mapper on gene expression data from breast tumors, Nicolau, Levine and Carlsson identified a previously unrecognized subgroup of breast cancers with high expression of the gene c-MYB and excellent survival. In the Mapper output this subgroup appeared as a separate branch, which standard clustering had not separated.

4.3.8 Neuroscience

Place cells. Certain neurons in the hippocampus of a rat, called *place cells*, fire when the animal is in a particular region of its environment, the *place field* of the cell. The discovery of place cells earned John O'Keefe a share of the 2014 Nobel Prize in Physiology or Medicine. The place fields form a cover of the environment, and a group of cells fires together exactly when the animal is in the intersection of their place fields. Hence the groups of co-firing cells are the simplices of the *nerve* of this cover. If the place fields and their intersections are contractible, the Nerve Theorem says that this nerve is homotopy equivalent to the environment itself. Curto and Itskov used this to show that the brain's firing patterns alone, with no access to the actual geometry, contain enough information to recover topological features of the environment, such as the presence of an obstacle in the middle of a room.

Clique topology. Giusti, Pastalkova, Curto and Itskov studied matrices of correlations between neurons. Instead of the actual values, which are distorted by unknown nonlinear effects, they used only the *order* of the correlations. Adding the edges in decreasing order of correlation gives a growing sequence of clique complexes, as for Vietoris-Rips. The number of holes in each dimension along this sequence distinguished correlations coming from an underlying geometric structure from random ones, using only the ordering of the matrix entries.

4.3.9 Biology: Evolution and Proteins

Viral evolution. When genes are passed only from parents to offspring, the history of a population is a tree, and a tree is contractible: it has no loops. But viruses exchange genetic material, by recombination in HIV and by reassortment of gene segments in influenza. Chan, Carlsson and Rabadan built Vietoris-Rips complexes from genetic distances between viral sequences, using only the distance matrix. Loops in these complexes indicate evolution that is not tree-like, so the 1-dimensional homology of the complex measures how much recombination and reassortment has occurred.

Protein pockets. A protein is modeled as a union of balls, one for each atom. Pockets and cavities in this union, where a drug molecule might bind, are 1- and 2-dimensional holes. Edelsbrunner and Mücke introduced **alpha shapes**, built from a subcomplex of the Delaunay triangulation, to compute such unions exactly and efficiently. Liang, Edelsbrunner and Woodward used this to measure protein pockets and cavities, and the approach underlies widely used software for locating binding sites.

4.3.10 Materials Science

Glass looks disordered: unlike a crystal, its atoms do not form a repeating lattice. Hiraoka, Nakamura, Hirata, Escolar, Matsue and Nishiura computed persistent homology of

the atomic positions in silica glass and other amorphous solids. The persistence diagrams separated liquid, glass and crystal states, and they revealed hierarchical ring structures in the glass that are invisible to the standard statistical descriptions used in physics.

4.3.11 Detecting Periodicity in Time Series

Let $f(t)$ be a signal measured over time and fix a delay $\tau > 0$. The **sliding window embedding** sends each time t to the point

$$SW f(t) = \big(f(t), f(t+\tau), f(t+2\tau), \ldots, f(t+d\tau)\big) \in \mathbb{R}^{d+1}.$$

If f is periodic, the curve $t \mapsto SW f(t)$ is closed, so the point cloud of all sliding windows lies on a loop. For example, with $f(t) = \sin t$, $d = 1$ and $\tau = \pi/2$, we get $SW f(t) = (\sin t, \sin(t + \pi/2)) = (\sin t, \cos t)$, which traces the unit circle. A non-periodic signal does not produce such a loop. Perea and Harer turned this into a periodicity score: the persistence of the most prominent 1-dimensional hole in the Rips complex of the sliding window point cloud. Because the score depends on the shape of the point cloud rather than on a particular frequency, it is robust to noise and to signals whose shape is not a pure sine wave.

4.3.12 Machine Learning

The output of persistent homology, a persistence diagram, is a finite collection of points in the plane, which is not directly usable by most machine learning algorithms. Methods such as *persistence landscapes* (Bubenik) and *persistence images* (Adams and coauthors) turn diagrams into vectors, so that topological features can be fed to standard classifiers together with other features. In the opposite direction, Naitzat, Zhitnikov and Lim used persistent homology to study what happens inside neural networks, observing that well-trained networks progressively simplify the topology of the data as it passes through the layers.

4.3.13 Theoretical Computer Science

Simplicial complexes also appear in pure computer science, where topology is used to prove that certain tasks are impossible or necessarily expensive. A common tool is the following combinatorial lemma.

Theorem 4.6 (Sperner's Lemma, dimension 2)**.** *Let a triangle with corners colored* 1*,* 2*,* 3 *be triangulated, and color every vertex of the triangulation with one of the colors* $1, 2, 3$ *so that a vertex on the side between the corners of colors* i *and* j *gets color* i *or* j*. Then the number of small triangles with all three colors is odd. In particular, there is at least one.*

Proof. Call an edge of the triangulation a *door* if its endpoints have colors 1 and 2. Doors on the boundary of the big triangle can only lie on the side between the corners 1 and 2. Walking along this side from corner 1 to corner 2, the color changes between 1 and 2 an odd number of times, so there is an odd number of boundary doors. Now count the pairs (small triangle, door of that triangle). A triangle with colors $\{1, 2, 3\}$ has exactly one door, a triangle with colors $\{1, 1, 2\}$ or $\{1, 2, 2\}$ has exactly two, and all other triangles

have none. On the other hand, each interior door lies in two small triangles and each boundary door in one. Hence

$$\begin{aligned}&\#\{\text{fully colored triangles}\} + 2\,\#\{\text{triangles with two doors}\}\\ &\quad = 2\,\#\{\text{interior doors}\} + \#\{\text{boundary doors}\},\end{aligned}$$

and the right-hand side is odd. □

The proof is also an algorithm: enter the big triangle through a boundary door and keep walking from triangle to triangle through doors. Every triangle has at most two doors, so the walk never revisits a triangle and must stop, either in a fully colored triangle or by leaving through another boundary door. Walks of the second kind pair up boundary doors, and there is an odd number of them, so at least one walk ends in a fully colored triangle. Path-following algorithms of this kind (Scarf) are used to compute approximate fixed points of continuous maps and equilibria in economics.

Fair division. Three roommates must split the rent of an apartment with three different rooms. Su showed how a Sperner-type labelling of a fine triangulation of the triangle of all possible rent splits leads, under mild assumptions on the roommates' preferences, to an (approximately) envy-free division: a split of the rent at which each roommate prefers a different room.

Distributed computing. In k-set agreement, each of $n+1$ processes running in parallel starts with an input value and must decide on one of the inputs, so that at most k different values are decided. In an asynchronous shared-memory system where any process may stop at any time (a "wait-free" protocol), this task turns out to be impossible for $k = n$. Herlihy and Shavit, and independently Saks and Zaharoglou and also Borowsky and Gafni, proved this by encoding all possible executions of a protocol as a simplicial complex (the *protocol complex*) which is a subdivision of a simplex, and the decisions as a Sperner coloring of it. Sperner's lemma then produces an execution in which all $n+1$ processes decide differently. The papers of Herlihy–Shavit and Saks–Zaharoglou received the 2004 Gödel Prize, one of the most prestigious awards in theoretical computer science.

Evasiveness. Suppose we want to decide whether an unknown graph on n vertices has some property, such as being connected, by asking questions of the form "is the edge $\{u, v\}$ present?". A property is *evasive* if every strategy must, in the worst case, ask about all $\binom{n}{2}$ edges. Kahn, Saks and Sturtevant proved that every nontrivial monotone graph property (one preserved by adding edges) is evasive when n is a prime power. Their key idea is that the graphs *without* the property, viewed as sets of edges, form a simplicial complex, since removing edges from such a graph keeps it without the property. They showed that a non-evasive property makes this complex contractible, and then a fixed point theorem for group actions on complexes gives a contradiction.

4.3.14 Software

Several open-source libraries implement the constructions of this course, and you are encouraged to try them as the semester progresses:

- **GUDHI** (C++ with a Python interface): simplicial complexes, Rips, Čech and alpha complexes, persistent homology.
- **Ripser** (C++, with a Python package): very fast persistent homology of Vietoris-Rips complexes.

- **giotto-tda** (Python): topological data analysis integrated with the `scikit-learn` machine learning library.

Writing your own small implementations first, as in the exercises below, is the best way to understand what these libraries do.

4.4 Summary

Application	Data	Complex	What topology gives
3D printing	triangle mesh	the mesh itself	closedness, genus, orientation
Mesh simplification	triangle mesh	the mesh itself	safe edge contractions (links)
Character recognition	pixels	cubical complex	components and holes
Sensor networks	who-hears-whom	Vietoris-Rips	certified coverage
Robotics	joint angles	configuration space	limits on motion planning
Natural images	3×3 patches in $\mathbb{R}^9$	Vietoris-Rips type	a Klein bottle
Mapper	any data set + a function	nerve of clusters	branches, subgroups
Place cells	co-firing neurons	nerve of place fields	shape of the environment
Viral evolution	genetic distances	Vietoris-Rips	non-tree-like evolution
Proteins	atom positions and radii	alpha complex	pockets, cavities
Glass	atom positions	alpha / Rips	ring structures
Time series	sliding windows	Vietoris-Rips	periodicity
Distributed computing	protocol executions	protocol complex	impossibility results

Programming Exercises

1. **A faster Rips complex.** The function `rips_complex` tests every subset of the points. Rewrite it so that a $(k+1)$-simplex is only built by adding a vertex to an existing k-simplex, and only vertices adjacent to all vertices of that simplex are tried. Compare the running times on 200 random points in the unit square for several values of ϵ.

2. **Inspecting a mesh.** Write a function that reads a triangle mesh in OBJ format and reports its numbers of vertices, edges and triangles, its Euler characteristic, the edges lying in only one triangle (holes) and the edges lying in three or more triangles (non-manifold edges). Run it on a mesh downloaded from the internet, and determine its genus if it is a closed orientable surface.

3. **Orientability.** Two triangles sharing an edge are *consistently oriented* if they traverse the shared edge in opposite directions. Write a program that tries to

orient all triangles of a closed surface consistently by a breadth-first search, flipping triangles when needed, and reports failure if a contradiction appears. Test it on the 9-vertex torus and the 6-vertex $\mathbb{R}P^2$.

4. **Components and Kruskal's algorithm.** Using a union-find data structure, compute the number of connected components of $VR_\epsilon(X)$ for every $\epsilon \geq 0$ by adding the edges in increasing order of length. Show that the values of ϵ at which two components merge are exactly half the lengths of the edges of a minimum spanning tree of X. This is the simplest case of persistent homology.

5. **Sliding windows.** Sample $f(t) = \sin t$ at 200 equally spaced times in $[0, 4\pi]$ and form the points $(f(t), f(t + \pi/2))$. Plot them, add random noise of increasing size, and use your Rips code to find the range of ϵ for which the complex has Euler characteristic 0 (as a circle does). Repeat with a non-periodic signal, such as $f(t) = t$.

6. **Covering an obtuse triangle.** Complete the proof of the proposition on sensor networks: if a triangle has an angle greater than $\pi/2$ and its sides have length at most r_b, then it is covered by the three disks of radius $r_b/\sqrt{3}$ around its vertices. Then check the proposition numerically by sampling random triangles and random points inside them.

Further Reading

- H. Adams et al., Persistence images: a stable vector representation of persistent homology, *J. Machine Learning Research* 18 (2017).

- U. Bauer, Ripser: efficient computation of Vietoris-Rips persistence barcodes, *J. Applied and Computational Topology* 5 (2021).

- A. R. Benson, R. Abebe, M. T. Schaub, A. Jadbabaie, J. Kleinberg, Simplicial closure and higher-order link prediction, *Proc. Natl. Acad. Sci. USA* 115 (2018).

- P. Bubenik, Statistical topological data analysis using persistence landscapes, *J. Machine Learning Research* 16 (2015).

- G. Carlsson, T. Ishkhanov, V. de Silva, A. Zomorodian, On the local behavior of spaces of natural images, *Int. J. Computer Vision* 76 (2008).

- J. M. Chan, G. Carlsson, R. Rabadan, Topology of viral evolution, *Proc. Natl. Acad. Sci. USA* 110 (2013).

- C. Curto, V. Itskov, Cell groups reveal structure of stimulus space, *PLoS Computational Biology* 4 (2008).

- V. de Silva, R. Ghrist, Coverage in sensor networks via persistent homology, *Algebraic & Geometric Topology* 7 (2007), 339–358.

- T. K. Dey, H. Edelsbrunner, S. Guha, D. V. Nekhayev, Topology preserving edge contraction, *Publ. Inst. Math. (Beograd)* 66(80) (1999), 23–45.

- H. Edelsbrunner, E. P. Mücke, Three-dimensional alpha shapes, *ACM Trans. Graphics* 13 (1994).

- M. Farber, Topological complexity of motion planning, *Discrete & Computational Geometry* 29 (2003).
- C. Giusti, E. Pastalkova, C. Curto, V. Itskov, Clique topology reveals intrinsic geometric structure in neural correlations, *Proc. Natl. Acad. Sci. USA* 112 (2015).
- M. Herlihy, D. Kozlov, S. Rajsbaum, *Distributed Computing Through Combinatorial Topology*, Morgan Kaufmann, 2013.
- Y. Hiraoka, T. Nakamura, A. Hirata, E. G. Escolar, K. Matsue, Y. Nishiura, Hierarchical structures of amorphous solids characterized by persistent homology, *Proc. Natl. Acad. Sci. USA* 113 (2016).
- J. Kahn, M. Saks, D. Sturtevant, A topological approach to evasiveness, *Combinatorica* 4 (1984).
- J. Liang, H. Edelsbrunner, C. Woodward, Anatomy of protein pockets and cavities: measurement of binding site geometry and implications for ligand design, *Protein Science* 7 (1998).
- G. Naitzat, A. Zhitnikov, L.-H. Lim, Topology of deep neural networks, *J. Machine Learning Research* 21 (2020).
- M. Nicolau, A. J. Levine, G. Carlsson, Topology based data analysis identifies a subgroup of breast cancers with a unique mutational profile and excellent survival, *Proc. Natl. Acad. Sci. USA* 108 (2011).
- J. A. Perea, J. Harer, Sliding windows and persistence: an application of topological methods to signal analysis, *Found. Computational Mathematics* 15 (2015).
- G. Singh, F. Mémoli, G. Carlsson, Topological methods for the analysis of high dimensional data sets and 3D object recognition, *Eurographics Symposium on Point-Based Graphics* (2007).
- F. E. Su, Rental harmony: Sperner's lemma in fair division, *Amer. Math. Monthly* 106 (1999).